ServiceNow GlideAjax

A Layman's Learning Guide: From Easy to Hard

This reference guide explains GlideAjax in plain English and walks through practical examples. Use this document to quickly build, debug, and understand your ServiceNow client-server integrations.

1. The Layman's Metaphor: The Restaurant Analogy

Imagine you are sitting at a table in a restaurant:
- You (The Customer): The Client Script (runs on your browser). You want information or want something done, but you can't walk into the kitchen yourself.
- The Kitchen (The Chef): The Script Include (runs on the ServiceNow server). The database lives here, and this is where the actual work/cooking happens.
- The Waiter: GlideAjax. The waiter takes your order, carries it to the kitchen, waits for the chef to cook it, and brings the food back to your table.

Why use a Waiter (GlideAjax)?
Without a waiter, you'd have to freeze the entire restaurant, walk into the kitchen, cook the food, and walk back. In web terms, doing things synchronously freezes the user's screen. GlideAjax lets the user keep typing or clicking while the server handles the request in the background (Asynchronously).

2. The Anatomy of GlideAjax (The Blueprint)

Every GlideAjax setup requires two components: the Server-Side Script Include (must be marked 'Client callable') and the Client-Side Client Script.

The Script Include (Server)

```
var MyCustomServerScript = Class.create();
MyCustomServerScript.prototype = Object.extendsObject(AbstractAjaxProcessor, {

    myServerFunction: function() {
        // 1. Get the information sent by the client
        var name = this.getParameter('sysparm_user_name');

        // 2. Do database query, calculations, etc.
        var result = "Hello " + name + " from the server!";

        // 3. Return the answer
        return result;
    },

    type: 'MyCustomServerScript'
});
```

The Client Script (Client)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    // 1. Summon the waiter and target the kitchen script
    var ga = new GlideAjax('MyCustomServerScript');

    // 2. Tell the waiter which function to call
    ga.addParam('sysparm_name', 'myServerFunction');

    // 3. Hand over custom parameters (must start with sysparm_)
    ga.addParam('sysparm_user_name', newValue);

    // 4. Send the waiter off and specify the callback function
    ga.getXMLAnswer(myCallBackFunction);
}

function myCallBackFunction(response) {
    g_form.addInfoMessage(response); // Prints the response from the server
}
```

NOTE: Always use 'sysparm_name' to define which function inside the Script Include you want to trigger. All other parameters you add must start with the prefix 'sysparm_'.

3. Practical Examples

Level 1: The Simple Greeting [Easy]

Goal: When a user changes a text field (u_nickname), send the nickname to the server and display a custom message below the field.

Script Include (Mark 'Client Callable' as True):

```
var GreetingAjax = Class.create();
GreetingAjax.prototype = Object.extendsObject(AbstractAjaxProcessor, {

    getGreeting: function() {
        var nickname = this.getParameter('sysparm_nickname');
        if (!nickname) {
            return "Hello Friend!";
        }
        return "Welcome back, " + nickname + "! Have a great day.";
    },

    type: 'GreetingAjax'
});
```

Client Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') return;

    var ga = new GlideAjax('GreetingAjax');
    ga.addParam('sysparm_name', 'getGreeting');
    ga.addParam('sysparm_nickname', newValue);
    ga.getXMLAnswer(displayGreeting);
}

function displayGreeting(answer) {
    g_form.showFieldMsg('u_nickname', answer, 'info');
}
```

Level 2: Fetching User Details [Medium]

Goal: When the caller changes on an Incident, fetch their email and phone from the server and auto-populate fields. Because we need multiple values back, we use JSON serialization.

Script Include (Mark 'Client Callable' as True):

```
var GetUserDetailsAjax = Class.create();
GetUserDetailsAjax.prototype = Object.extendsObject(AbstractAjaxProcessor, {

    getUserContactInfo: function() {
        var userId = this.getParameter('sysparm_user_id');
        var responseObj = { email: '', phone: '' };

        var userGR = new GlideRecord('sys_user');
        if (userGR.get(userId)) {
            responseObj.email = userGR.getValue('email') || 'No Email';
            responseObj.phone = userGR.getValue('phone') || 'No Phone';
        }
        return JSON.stringify(responseObj);
    },

    type: 'GetUserDetailsAjax'
});
```

Client Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') return;

    var ga = new GlideAjax('GetUserDetailsAjax');
    ga.addParam('sysparm_name', 'getUserContactInfo');
    ga.addParam('sysparm_user_id', newValue);
    ga.getXMLAnswer(populateContactDetails);
}

function populateContactDetails(answer) {
    if (answer) {
        var userInfo = JSON.parse(answer);
        g_form.setValue('email', userInfo.email);
        g_form.setValue('u_phone', userInfo.phone);
    }
}
```

Level 3: Validation & Complex Check [Hard]

Goal: Before assigning a Change Request to a user, check if they have active Critical (P1) or High (P2) incidents, and display an error message warning the user.

Script Include (Mark 'Client Callable' as True):

```
var AssignedUserCheckAjax = Class.create();
AssignedUserCheckAjax.prototype = Object.extendsObject(AbstractAjaxProcessor, {

    checkOpenIncidents: function() {
        var assignedTo = this.getParameter('sysparm_assigned_to');
        var result = {
            hasCriticalIncidents: false,
            incidentCount: 0,
            latestIncidentNumber: ''
        };
        if (!assignedTo) return JSON.stringify(result);

        var incidentGR = new GlideRecord('incident');
        incidentGR.addActiveQuery();
        incidentGR.addQuery('assigned_to', assignedTo);
        incidentGR.addQuery('priority', 'IN', '1,2');
        incidentGR.orderByDesc('sys_created_on');
        incidentGR.query();

        result.incidentCount = incidentGR.getRowCount();
        if (result.incidentCount > 0) {
            result.hasCriticalIncidents = true;
            if (incidentGR.next()) {
                result.latestIncidentNumber = incidentGR.getValue('number');
            }
        }
        return JSON.stringify(result);
    },

    type: 'AssignedUserCheckAjax'
});
```

Client Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        g_form.clearMessages();
        return;
    }

    var ga = new GlideAjax('AssignedUserCheckAjax');
    ga.addParam('sysparm_name', 'checkOpenIncidents');
    ga.addParam('sysparm_assigned_to', newValue);
    ga.getXMLAnswer(handleIncidentCheckResult);
}

function handleIncidentCheckResult(answer) {
    g_form.clearMessages();
    if (answer) {
        var result = JSON.parse(answer);
        if (result.hasCriticalIncidents) {
```

```
            var msg = "[WARNING] This user has " + result.incidentCount +
                      " open High/Critical incident(s). Most recent: " + result.latestIncidentNumber;
            g_form.addErrorMessage(msg);
        }
    }
}
```

4. Key Takeaways & Best Practices

1. getXMLAnswer() is Better:
Always use getXMLAnswer() instead of getXML(). It returns just the string value inside the XML envelope directly. This makes client-side parsing clean and simple.

2. Use JSON Strings for Objects:
When returning multiple columns or details from the server, save them in a standard JS object, run JSON.stringify(object) on the server, and parse it on the client with JSON.parse(answer).

3. Client Callable Is Mandatory:
Double-check the 'Client Callable' checkbox on the Script Include record. If it is unchecked, the GlideAjax call will silently fail or return null.

4. Keep Script Includes Small and Pure:
Write single-purpose functions in your Script Includes to keep the server code clean and fast.

5. Debugging Guide

- Browser Inspector: Press F12, look at the Console tab. Add jslog() statements before sending GlideAjax and inside the callback to trace data.
- System Logs: Check 'syslog' in ServiceNow. Put gs.info() inside your Script Include to see if the server script is actually being entered and what parameters it reads.
- Network Tab: Look for 'xmlhttp.do' network requests in browser developer tools to verify the exact payload sent and returned.